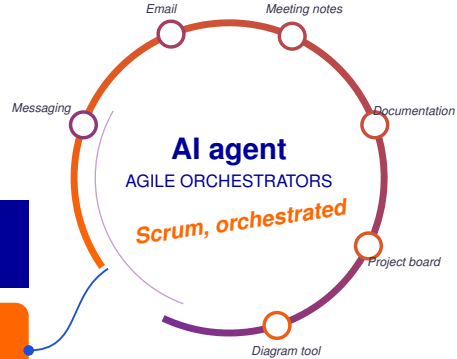
Innovation Challenge

Agile Orchestrators

AI agents for Agile ways of working



Adam Alansary · Cedric Baron · Jasper Krebbers · Theun van Vliet · Stefan Wachmann

Rabobank

The challenge

Situation

- Rabobank teams work in an Agile way
- The context of that work is spread over repositories, documentation, tools and the project board
- AI tools are available, but with limited context and integration

Complication

- Agile chores take time: writing stories, refinement, status updates
- With context in many places, there is no single source of truth, so stakeholders and users get conflicting information
- AI tools don't know our backlog, conversations, documentation, code or agreements

Question

How can AI agents contribute to our Agile workflow to improve quality and reduce friction?

A safe place to try it

Our test case: Tulip Bank

To try this safely, we set up a realistic project at Tulip Bank, a fictional bank that works the way Rabobank does.

- The project: a churn model with a scoring API, built on synthetic data
- It is handed over to a new team: **us**

It is set up like a real Scrum team

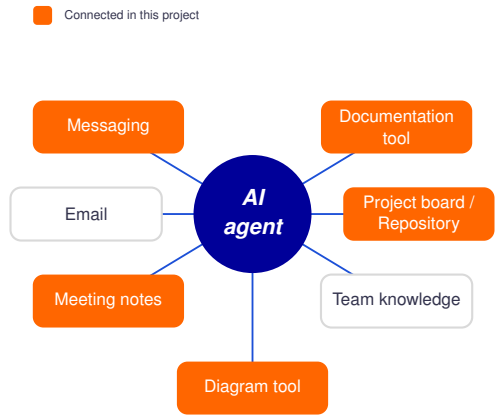
At Tulip Bank	At Rabobank	Used for
Claude Code, Codex	GitHub Copilot	AI agent
GitHub Project	Azure DevOps	Backlog, sprints and code
GitHub Wiki	Confluence	Documentation and meeting notes
Slack	Teams	Messaging
Miro	Miro	Diagrams and the retro board

Getting started: giving the agent context

We take over the project. What is the situation, what are the priorities, what needs to get done?

- We ask the existing agent for an overview of the current repository and information we have
- The first answers have gaps: the agent can't reach all sources, and some information isn't written down anywhere

This is our solution



Recurring work, written down once

What is a Skill?

Now that the agent is connected to our tools, it can find the information. Next it needs to know **how our team works**: how we write PBIs, our best practices, how refinement and planning usually go. We wrote this down as **Skills**, co-created with our agent.

A Skill

- Is a work instruction for the AI agent, for one task: “create a PBI”, “run refinement”
- Holds the team’s standards: Definition of Ready, templates, board fields
- Keeps the output standardized and consistent
- Is plain text in the repository, so the team reviews and improves it like code
- The agent detects when a Skill is needed, and only then reads the full instructions
- Can also be manually triggered as a slash command

Example, shortened from `create-pbi/SKILL.md`

```
---
name: create-pbi
description: Create a Product
  Backlog Item as a GitHub issue with
  the correct type, labels, parent
  link and board fields. Use when
  asked to create a PBI.
---

1. Check the Definition of Ready
2. Draft the body
3. Confirm
4. Create
5. Verify and report
```

Preparing the sprint

We go from business goal to a planned sprint, with one Skill per step.



Goal to epic

`create-epic`

Context it uses

Business goal on the wiki

Epic to features

`create-features`

Context it uses

Epic on the board

Feature to PBIs

`create-pbis`

Context it uses

Feature, Definition of Ready

Refinement

`refinement-session`

Context it uses

Items in refinement, Definition of Ready

Sprint planning

`sprint-planning`

Context it uses

Ready items, capacity, deadlines

Every Skill is interactive: the agent proposes, a person decides

Team composition and their capabilities turned out to be important context as well

Doing the work

With the sprint planned, we start building. Along the way, the usual chores show up: bugs we run into, a task board to keep up to date, outdated documentation and pull requests to review.

01 → 02 → 03 → 04 → 05

01 Build

`work-on-pbi`

Context it uses

PBI, code, Definition of Done

02 Verify and publish PR

`verify-publish-pr`

Context it uses

Acceptance criteria, code changes, Definition of Done

03 Review PR

`review-pr`

Context it uses

PR, interactive questions

04 Log a finding

`log-finding`

Context it uses

The finding, the board

05 Update the docs

`post-merge-docs`

Context it uses

The merged PR, its diff, the docs and the wiki

`log-finding` *puts bugs and other findings on the board right away, without asking for an OK*

On its own, the agent's review was too shallow, so we review pull requests together with the agent

`post-merge-docs` *keeps the docs accurate after a merge, without guessing at behavior or publishing live wiki edits without approval*

Closing the sprint

At the end of the sprint, we wrap up, show what we built and look back.

01 02 03

Wrap up

`ending-the-sprint`

Context it uses

Open items on the board

Sprint review

`create-sprint-overview`

Context it uses

Finished work on the board

Retrospective

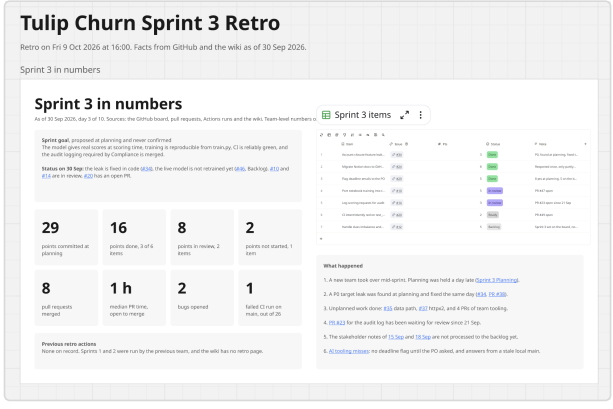
`create-retrospective`

Context it uses

Board, repository, meeting notes

`create-sprint-overview` *builds the internal sprint review presentation, including suggested next steps*

Example: retrospective



Prompt

Start the retro.

Agent

The retro board is ready in Miro. I've added talking points from the sprint, the sprint's metrics and links to the project board for quick access.

The agent does the preparation, so the team can spend the retro on the conversation

Lessons that carry over

What we learned and what's next

Tulip Bank works the way Rabobank does, so these lessons carry over directly.

What we learned

- Do a task once with the agent; if it is recurring, turn it into a Skill
- Give the agent access to all relevant context, and let it keep that context up to date
- Keep people in control: make Skills interactive and easy to use
- The agent can prepare our rituals, and even run them temporarily: setting up the board, sending invitations, adding quick links

What's next

- Allow us to connect our tools to our AI agents
- Give people room to experiment with new integrations
- Offer approved integrations in a marketplace
- Start a shared Skill library where teams find, reuse and improve each other's Skills
- Standardize connectors across agentic interfaces

Appendix: slopguard

Agents write code fast, but not always clean code. **slopguard** is a Claude Code hook that keeps the agent from finishing its turn while the code contains AI slop, and tells it what to fix.

What it blocks

Too complex	branching, arguments, nesting (ruff)
Too long	files over a line limit
Type slop	deeply nested annotations, wide tuples
Dead code	anywhere in the project (vulture)
Copy-paste	duplicated blocks (PMD CPD)
Never loops	lets go after one fix attempt

github.com/kimzed/python-slopguard

Our setup on Tulip Churn, from `pyproject.toml`

```
[tool.slopguard.edit]
max_complexity = 4
max_args = 4
max_statements = 15
max_file_lines = 400
existing_violations = "block"

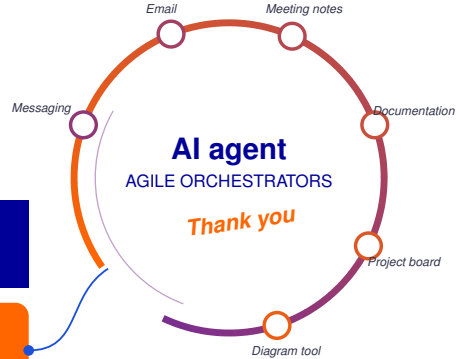
# install once, then per project
uv tool install \
  git+https://github.com/kimzed/python-slopguard
slopguard init
```

Innovation Challenge

Questions?

Thank you from the Agile Orchestrators

Adam Alansary · Cedric Baron · Jasper Krebbers · Theun van Vliet · Stefan Wachmann



Rabobank